PayLink Midterm Submission and Recording Guide

Sai Kadamati and Siddhardth

Recording readiness

The public app is live at https://paylink-radiant690s-projects.vercel.app. Its path is Next.js on Vercel, FastAPI on Render, and the private five-table PostgreSQL schema on Supabase in primary-org/paylink. Nine hosted browser acceptance groups passed on this Mac, including actual committed transfers and zero balance reconciliation differences. The Mac also passed 25 database/API tests and 12 local browser workflow groups. See validation.md for evidence and boundaries.

One representative records one video for the team; one representative submits it. The script below is designed for about eight minutes. This duration is a practical suggestion, not a claim about an instructor-imposed limit. Keep the public app and the actual Supabase project visible during the database demo. No real money or bank integrations are involved.

Private synthetic profile passwords are stored only in the ignored .local/recording-users.json on the setup Mac. Open that file privately to sign in as sai and siddhardth. Do not put it in the submission, show it on video, or use the public local-only demo password on the hosted app. Use a normal Chrome window for Sai and an Incognito window for Siddhardth so their cookies stay separate.

Submission package

Submit the video and the deliverables requested by the instructor: application source, the midterm report, database design analysis, editable ER model, and supporting SQL/evidence. The Markdown sources have corresponding PDF, Word, and HTML exports under docs/. The native editable ER source is er-diagram.drawio; er-diagram.mmd and er-diagram.dot are alternate editable sources. A PNG alone is not an editable model.

The source repository is https://github.com/venkatasai-kadamati/adb_paylink on main. Validation artifacts distinguish local test fixtures, which were cleaned up, from hosted synthetic recording rows, which remain available. Backup/restore, crash recovery, and production-scale load testing have not been demonstrated.

Proposed contribution split

Codex generated and revised the implementation and supporting materials. This table proposes responsibility for review, explanation, follow-up work, and presentation. It does not assign historical authorship. Each teammate should replace proposed ownership with a truthful statement of their actual work before submitting.

Area	Sai proposed review responsibility	Siddhardth proposed review responsibility
Requirements and modeling	Financial invariants, candidate keys, functional dependencies	ER cardinalities, ownership bridge, relational mapping
Database	DDL, constraints, transfer locks, idempotency, restricted role	Dictionary, synthetic catalog, joins, aggregates, running balances
Application	FastAPI identity, parameterized SQL, error and rollback paths	Next.js flows, joint ownership, history, CSV, insights, mobile layout
Verification and delivery	Integrity/concurrency tests, reconciliation, EXPLAIN	Browser evidence, screenshots, report, video, hosted acceptance

Safe narration: “Sai’s proposed review focus is the transaction and backend layer; Siddhardth’s is the interface and relationship documentation. We will attribute individual contributions only after each teammate confirms the work they performed.” If actual review and changes are confirmed, name the concrete work and evidence instead. Do not say either teammate independently implemented the generated code.

Four incremental milestones

1. **Domain and design.** Define the simulation, five entities, one-to-many bank/account relationship, many-to-many ownership, candidate keys, and normalization assumptions. Evidence: editable ER diagram and database-design analysis. The first increment explains the model before implementation.
2. **Reliable database layer.** Add versioned migrations, PK/FK/UNIQUE/CHECK constraints, synthetic bank catalog, atomic account opening, joint-owner grants, and the transfer function. Evidence: runnable SQL, limited backend role, and reconciliation. This builds a database that can enforce its supported invariants.
3. **End to end application.** Connect FastAPI sessions to Next.js registration, accounts, exact recipient lookup, transfer review and confirmation, own-account movement, history, CSV, and insights. Evidence: real user interactions and persisted database rows.
4. **Validation and midterm delivery.** Run tests on this Mac, inspect live screenshots, compare query plans, publish the Vercel/Render/Supabase path, and assemble reports and recording materials. Evidence: 25 database/API tests, 12 local browser groups, nine hosted browser groups, and zero reconciliation discrepancies. The proposed individual review assignments still require team confirmation.

Future milestones 5 through 7

- **5 Hosted operations and recovery.** Extend completed hosted acceptance with sustained hosted concurrency, monitoring, a backup/restore drill, and checks after redeploy and free-service wakeup. The hosted schema and basic public transfer path already work.
- **6 Account and audit model.** Add invitation acceptance and coordinated owner revocation, pagination, reversals, and an evaluated immutable double-entry ledger. Immediate equal-owner grants are implemented; invitation/revocation and a ledger are future work.

- **7 Scale and final evaluation.** Test representative load and skew, repeat query-plan measurements, validate multi-worker security and limiting, refine accessibility, and prepare the final course evaluation with measured evidence.

Video script and screen cues

The script assumes Sai narrates the whole video. If Siddhardth records, replace only the introduction name and any confirmed personal-contribution statements. The remaining script is ready to rehearse; adapt only confirmed personal-contribution statements. Screen directions are not spoken.

0 minutes to 1 minute Introduction and project

“Hello, I’m Sai Kadamati, presenting our CS 5600 Advanced Database Systems midterm update. Our teammates are Sai Kadamati and Siddhardth. Our project is PayLink, Simulated Peer-to-Peer Banking Transfers.

“PayLink lets registered users create simulated checking and savings accounts, share an account with another registered user, find recipients, send funds, move funds between their own accounts, and inspect history and insights. It never connects to real banks or moves real money. The main database challenge is preserving both balances and the transfer record when requests overlap, fail, or repeat.”

Show the report title, then the public app’s simulation label and account overview.

1 minute to 2 minutes Four milestones and team responsibilities

“Our first milestone defined the domain, ER model, keys, and normalization assumptions. The second built the five-table schema, constraints, account functions, and atomic transfer function. The third connected those rules to FastAPI authentication and the Next.js interface. The fourth tested the complete system on this Mac, published it with Supabase, and prepared the midterm evidence.

“Sai’s proposed review focus is the database transaction and backend layer. Siddhardth’s proposed focus is the interface, relationship documentation, and browser demonstration. These are review assignments, not a claim of independent implementation. Codex assisted with code, design documents, tests, deployments, and recording materials. We will report each teammate’s actual contributions accurately.”

Show the four milestones and contribution table. If both teammates have confirmed actual work, replace the proposed-focus sentences with those facts. Keep the AI disclosure accurate.

2 minutes to 3 minutes Relational design and normalization

“There are five core tables: Users, Banks, BankAccounts, AccountHolders, and Transfers. A bank maintains many accounts. Users and accounts have a many-to-many relationship, represented by AccountHolders with a composite user and account key. Two joint owners therefore refer to the same account and balance.

“Transfers has separate source and destination account foreign keys, plus an initiating user. Account numbers are unique within a bank. A transfer’s idempotency key is unique within its initiating user.

“We separated user, bank, account, ownership, and event facts to remove partial and transitive dependencies. For example, account ID determines bank ID, and bank ID determines bank details. Under the stated dependencies, each nontrivial determinant in a final relation is a superkey, giving BCNF. Ownership permission and sufficient funds remain transaction rules; normalization alone does not enforce them.”

Open the editable diagram in diagrams.net. Point to the bridge, composite key, and the two account roles. If time is short, show the exported diagram and identify the editable source.

3 minutes to 4 minutes Transactions and database implementation

“Money is stored as integer USD cents. The backend derives the actor from a verified signed session cookie. It checks source ownership, locks both accounts in UUID order, checks active state and funds, debits one account, credits the other, and inserts one completed event in a single transaction. Success is returned after commit; failure rolls back the whole operation.

“READ COMMITTED plus those row locks serializes conflicting balance changes for this workflow. Consistent account ordering prevents the classic opposing-transfer deadlock. An actor-scoped advisory lock and unique idempotency constraint serialize retries: identical details return the original transfer, while changed details with the same key are rejected.

“The backend database role cannot directly update balances or insert transfers. Controlled functions are in a private schema with fixed search paths and no PUBLIC execution. Joint-owner grants lock the target user before the account and enforce the same four-account limit as account creation.”

Show the relevant SQL function and the current 25-test Mac result. Explain one rollback or concurrency test that the narrator can actually describe.

4 minutes to 6 minutes Public UI and database demo

“I’m signed in as Sai on our public Vercel site. The API runs on Render and the data is stored in Supabase. These are synthetic accounts; creating one allocates one thousand simulated dollars. Shared expenses has two owners but one balance.

“I’ll send twelve dollars and thirty-four cents to Siddhardth. Recipient lookup uses an exact username, email, or phone. The review shows the source, destination, amount, and remaining balance. Confirming gives us a transfer ID.

“Using that exact ID in Supabase, we can see one transfer row. The sender is lower by one thousand two hundred thirty-four cents and the recipient is higher by the same amount. Reloading preserves the receipt. In Siddhardth’s separate session, it appears as received. I can also move funds from an account he jointly owns.

“An insufficient-funds attempt is rejected. Our automated hosted check additionally retried the same idempotency key and confirmed one payment, and rejected a non-owner source and changed payload without further balance changes.”

Before sending, record both actual balances. Afterward, copy the new transfer ID. In the Supabase SQL editor use docs/recording-demo.sql, replacing the example ID with the new receipt ID. Show the event and both accounts; do not read a prepared before/after number if you sent an additional

payment. Show history, CSV download, and Insights briefly. Avoid password fields, connection strings, role credentials, and cookies on screen.

6 minutes to 7 minutes SQL evidence and validation

“We use joins for bank and ownership details, aggregation for account and insight totals, a window function for running balances, and a reconciliation query that compares stored balances with opening balances plus incoming minus outgoing transfers. The hosted reconciliation returns zero differences.

“On this Mac, twenty-five database and API tests passed, including concurrent retries, competing spending, opposing transfers, authorization, foreign-key lock compatibility, constraints, and rollback. Twelve local browser groups and nine hosted browser groups passed with no JavaScript errors. A local twenty-thousand-row temporary benchmark showed the source/time index avoiding a full scan and sort; those fixture rows and the temporary index removal were rolled back. The timings describe one local run, not hosted performance.”

Run the read-only reconciliation query in Supabase. Show artifacts/test-results.txt and hosted-browser-results.json. Use the current query-evidence.txt if discussing exact timings. Do not claim backup restoration or production-scale load tests were completed.

7 minutes to 8 minutes Future milestones and close

“Milestone five focuses on hosted operations, representative concurrency, monitoring, and a backup/restore drill. Milestone six extends account invitations and revocations, pagination, reversals, and the audit or ledger model. Milestone seven measures scale, revisits query optimization, improves accessibility, and completes the final evaluation.

“Our current demonstration covers the public application and a real hosted PostgreSQL transaction. All balances and payments are simulated. Thank you.”

Practical recording checklist

- Open the public URL and its /api/health first. Allow Render’s free service to wake up, then verify login before starting the recording.
- Keep Sai and Siddhardth in separate browser sessions. Retrieve their private synthetic passwords from the ignored local file before sharing or recording the screen.
- Open the editable ER diagram, SQL function, current test output, and Supabase SQL editor in advance.
- Have docs/recording-demo.sql ready. Copy the actual new receipt ID after the recorded payment.
- Show before and after balances, then reconciliation. Additional transfers change the values; never present old snapshots as current balances.
- Record one video with one representative. Name both teammates and all four milestones, then future milestones 5 through 7.
- Replace proposed contribution statements only with work each teammate confirms. Preserve the accurate Codex disclosure and follow the instructor’s AI policy.